RutaIA
TURISMO INTELIGENTE PARA COLOMBIA

PROYECTO FINAL · PROGRAMACIÓN COMERCIAL

RutaIA

Turismo con IA, agentes multi-paso y control de tokens

React + Vite · Express · MySQL 8 · Groq · Power BI
Repositorio: github.com/FerMar707501/PFinalTurismo
API: http://localhost:4000 · Swagger: /api/docs

Documentación técnica y funcional

Entrega académica · Noviembre de 2026

1. Resumen del proyecto

RutaIA es una aplicación web de turismo colombiano que combina un catálogo administrable con tres servicios de inteligencia artificial: un chatbot con generación aumentada por recuperación (RAG), un agente planificador de itinerarios y un agente comparador de destinos. La plataforma autentica por roles, limita el consumo mensual de tokens por usuario, registra cada ejecución y expone un reporte analítico compartido por el panel web y Power BI.

- Chat RAG: recuperación FULLTEXT sobre 60 documentos, memoria de la conversación y fuentes citadas.
- Agentes multi-paso: tool calling con Groq, hasta seis pasos y herramientas internas y externas.
- Control de cuota: estimación previa, bloqueo con HTTP 429 y registro del consumo real posterior.
- Administración: usuarios y cuotas, destinos, actividades, servicios de IA y claves de API.
- Analítica: un mismo agregado para la web y Power BI, con exportación CSV.
- Calidad: 68 pruebas de integración, Swagger, colección Postman y CI/CD en GitHub Actions.

Arquitectura de la solución



Capa	Tecnología	Responsabilidad
Frontend	React 19 + Vite	SPA con navegación por hash, roles, chat, agentes y administración
API	Express 5 (Node 22)	Rutas REST, validación Zod, JWT, cuotas, SSE de errores y OpenAPI
Datos	MySQL 8.4	Catálogo, usuarios, conversaciones, uso, trazas y caché externa
IA	Groq (gpt-oss-120b/20b)	Chat con contexto, decisiones multi-paso y tool calling
Fuentes abiertas	Open-Meteo · OSRM · Wikipedia	Clima, rutas y contexto cultural sin claves
BI	Power BI Desktop	Reporte de consumo con API key y refresco programado
Despliegue	Render · Vercel · GitHub Actions	CI con pruebas y despliegue condicionado

2. Requisitos y alcance

Requisito del curso	Implementación en RutaIA	Evidencia
Chatbot con RAG y memoria	POST /api/chat con recuperación FULLTEXT de 5 documentos y últimos 10 mensajes	Código, pruebas Fase 4 y captura 04
Dos servicios de IA adicionales	Agentes de itinerario y comparación con hasta 6 pasos y 5 herramientas	Pruebas Fase 5 y capturas 05-06
Login con roles	JWT, bcrypt y roles admin/user; sin registro público	Pruebas Fase 3 y captura 01
CRUD administrativo	Usuarios, destinos, actividades y servicios de IA	Capturas 10-13
Límite de tokens	Cuota mensual, reserva previa, bloqueo 429 y registro real	Fase 4-5 y captura 08
Conversaciones e historial	Conversaciones por usuario, mensajes, fuentes y ejecuciones de agentes	Capturas 04 y 07
API REST con permisos	26 rutas documentadas, JWT, API keys y CORS	Swagger y Postman
Power BI conectado	Endpoint externo con X-API-Key y mismo agregado	powerbi/README y captura 18
Pruebas automatizadas	68 pruebas de integración con Vitest y Supertest	CI en verde
Despliegue y CI/CD	Render, Vercel y GitHub Actions con smoke test	Workflows ci.yml, smoke.yml y CodeQL

Quedan fuera del alcance del curso: pagos, reservas reales, mapas interactivos y aplicaciones móviles nativas. El diseño mantiene una ruta de crecimiento clara hacia esos módulos sin cambiar el modelo de datos principal.

3. Modelo de datos

El esquema usa utf8mb4_spanish_ci, almacena fechas en UTC y aplica claves foráneas para preservar la trazabilidad. El DDL completo está en `db/schema.sql` y los datos de demostración en `db/seed.sql`.

Tabla	Propósito	Claves y relaciones
users	Usuarios, rol y cuota mensual	PK id · UQ email
destinations	Catálogo turístico administrable	PK id · índice nombre/región
activities	Oferta de cada destino	FK destination_id → destinations
documents	Fragmentos con índice FULLTEXT para el RAG	source_type/source_id
ai_services	Modelo, temperatura, tokens y prompt	PK code
conversations / messages	Historial del chat y de los agentes	FK user_id y conversation_id
usage_logs	Hechos de consumo para cuotas y analítica	FK user_id, service_code, conversation_id
agent_runs	Entrada, salida, pasos y tokens por ejecución	FK user_id · service_code
api_clients	Claves de API con hash y scope	PK id · UQ key_hash
external_cache	Caché de clima, rutas y Wikipedia	PK cache_key · expires_at

Los datos de demostración contienen 3 usuarios, 12 destinos, 48 actividades, 3 conversaciones, 6 mensajes, 22 registros de consumo, 3 servicios de IA y 60 documentos. La ingesta se puede repetir con `npm run db:ingest`.

Vistas de apoyo

v_usage_daily y v_usage_by_service resumen el consumo para consultas rápidas; la API de analítica usa las tablas base con parámetros para respetar el rango y el filtro por servicio sin depender de las vistas.

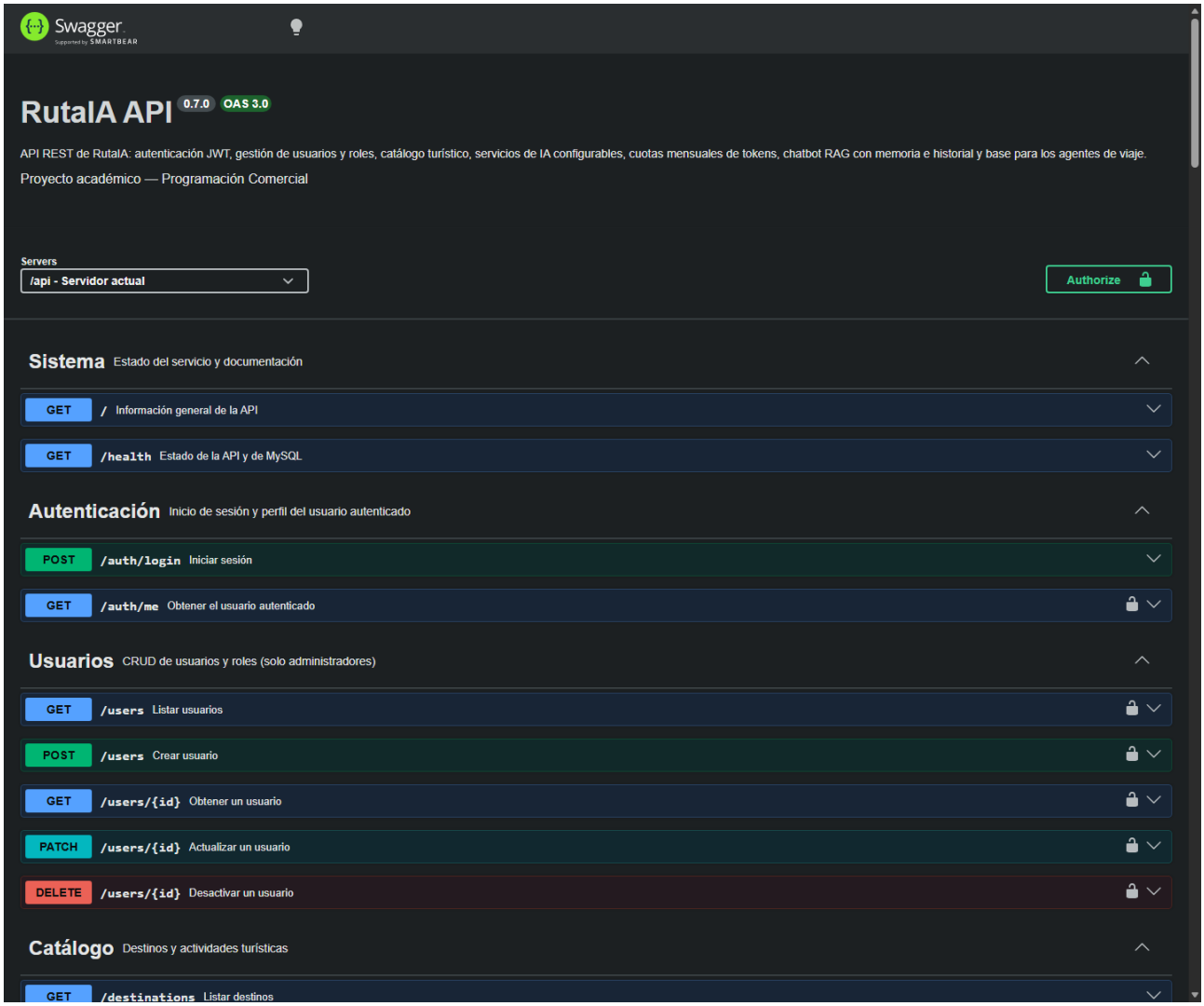
4. API REST y documentación

La API está documentada en OpenAPI 3.0.3 y disponible en http://localhost:4000/api/docs. La autenticación principal es JWT; los consumidores externos usan una API key con scope `analytics:read`.

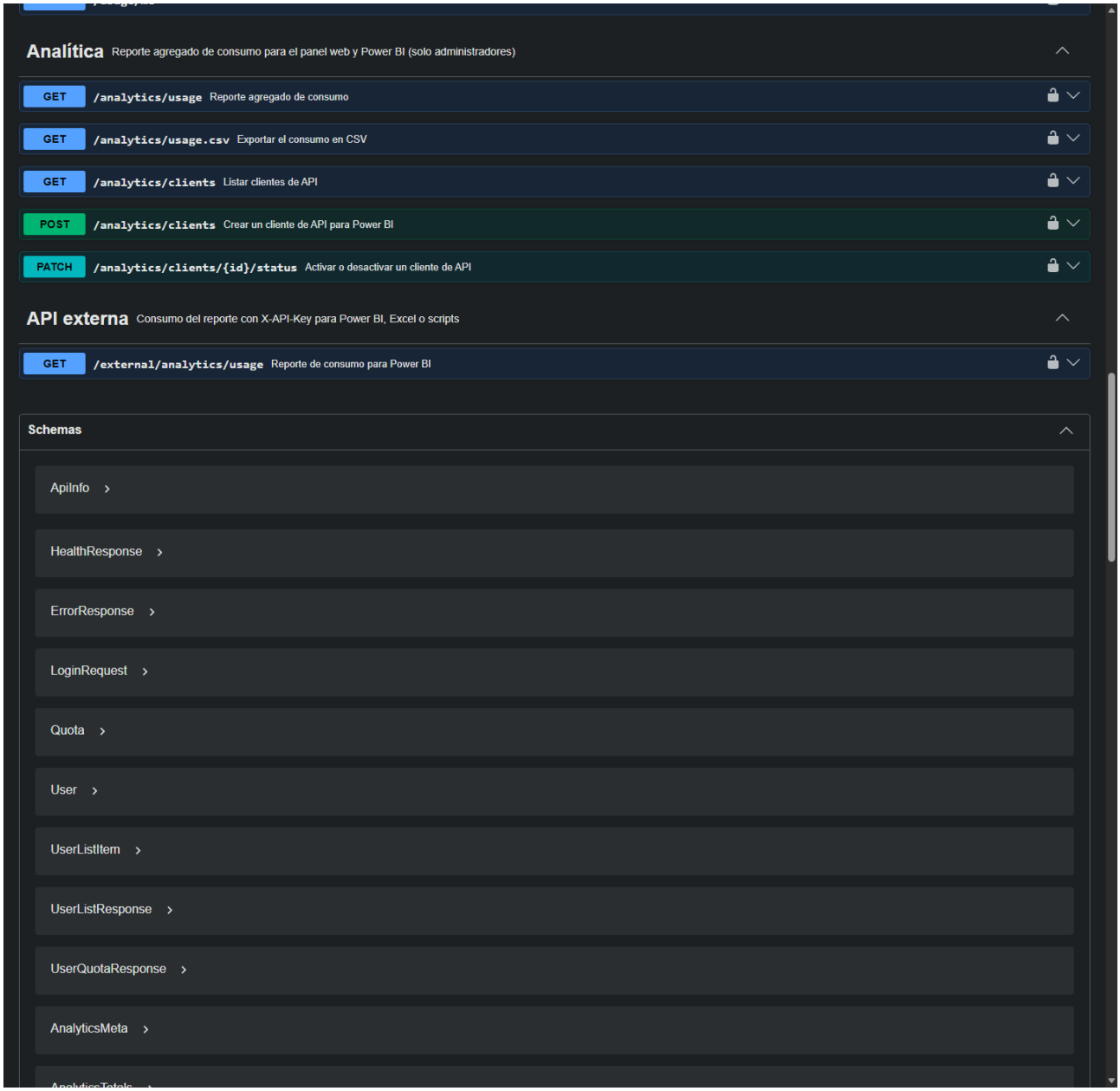
Método y ruta	Autenticación	Descripción
POST /api/auth/login	Pública	Entrega JWT y cuota del usuario
GET /api/auth/me	JWT	Perfil y cuota vigente
GET/POST /api/users	JWT admin	Listado y creación de usuarios
PATCH/DELETE /api/users/:id	JWT admin	Edición y baja lógica
GET /api/destinations, /:id	Pública	Catálogo y ficha con actividades
POST/PATCH/DELETE /api/destinations	JWT admin	CRUD y sincronización del RAG
GET /api/activities	Pública	Actividades con filtros
POST/PATCH/DELETE /api/activities	JWT admin	CRUD y sincronización del RAG
GET /api/services	Pública/JWT	Servicios activos; admin ve todos
PATCH /api/services/:code(/status)	JWT admin	Configuración y activación
GET /api/usage/me	JWT	Cuota mensual del usuario
POST /api/chat	JWT	Mensaje al chatbot RAG
GET /api/chat/conversations(/:id)	JWT	Historial y fuentes propias
POST /api/agents/itinerary	JWT	Agente de itinerarios
POST /api/agents/compare	JWT	Agente comparador
GET /api/agents/runs(/:id)	JWT	Trazabilidad de ejecuciones
GET /api/analytics/usage(.csv)	JWT admin	Reporte agregado y exportación
GET/POST/PATCH /api/analytics/clients	JWT admin	Gestión de API keys
GET /api/external/analytics/usage	X-API-Key	Reporte para Power BI

Códigos de error relevantes

HTTP	Código	Cuándo ocurre
400	VALIDATION_ERROR	Datos inválidos o rango de fechas incorrecto
401	INVALID_TOKEN / API_KEY_REQUIRED	JWT inválido o falta X-API-Key
403	FORBIDDEN / USER_INACTIVE	Sin rol admin o cuenta desactivada
409	SERVICE_DISABLED / EMAIL_IN_USE	Servicio apagado o correo repetido
429	TOKEN_QUOTA_EXCEEDED	La reserva de tokens supera la cuota
503	AI_NOT_CONFIGURED	No está configurada la clave de Groq



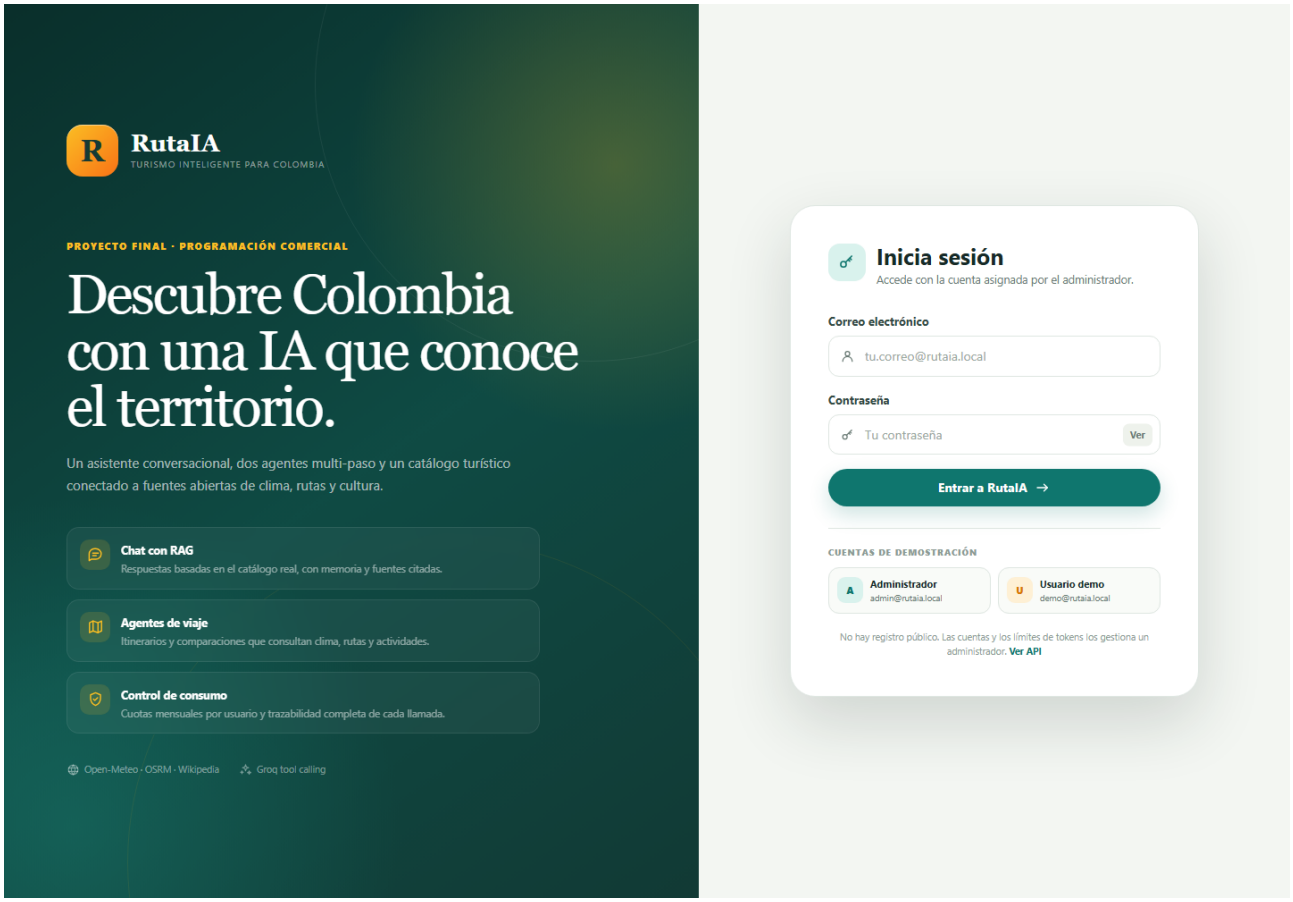
Swagger UI con los grupos de la API



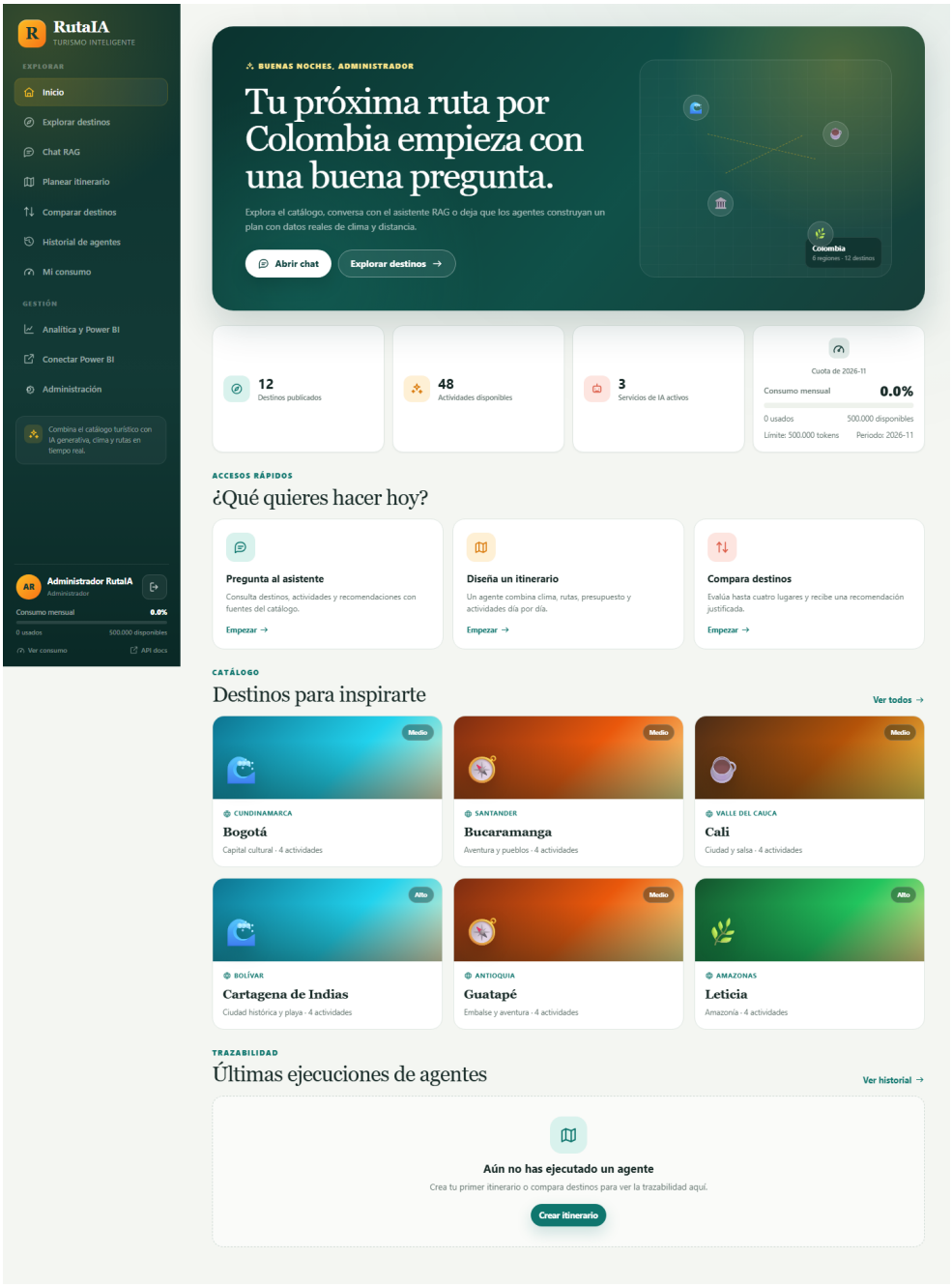
Endpoints de analítica y API externa en Swagger

5. Aplicación web

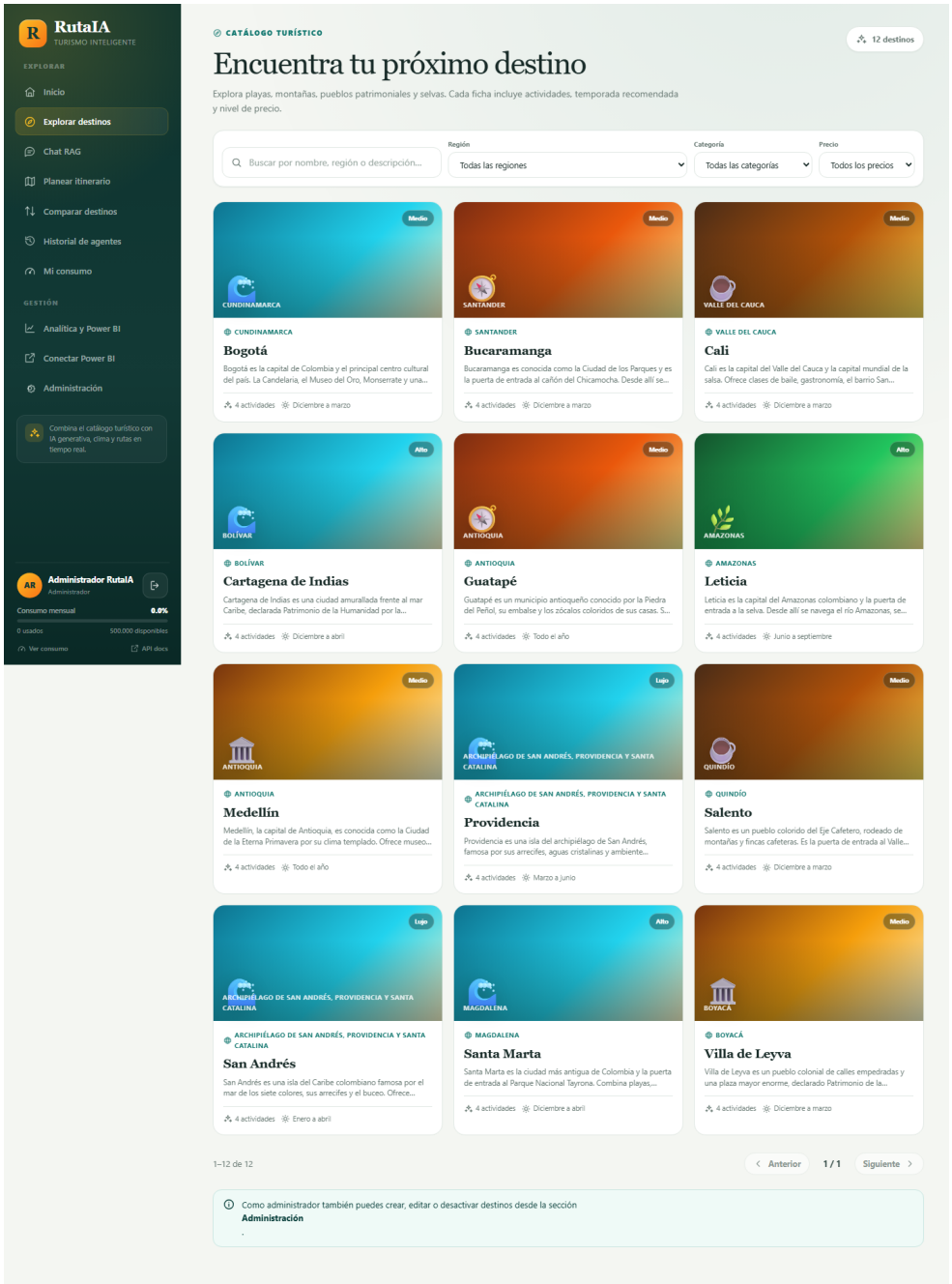
El frontend se construyó con React 19 y Vite, sin librerías adicionales de UI ni de Markdown. La navegación es por hash, el estado de sesión se guarda en localStorage y cada llamada envía el JWT. El diseño es responsive desde 320 px.



Pantalla de inicio de sesión con cuentas de demostración



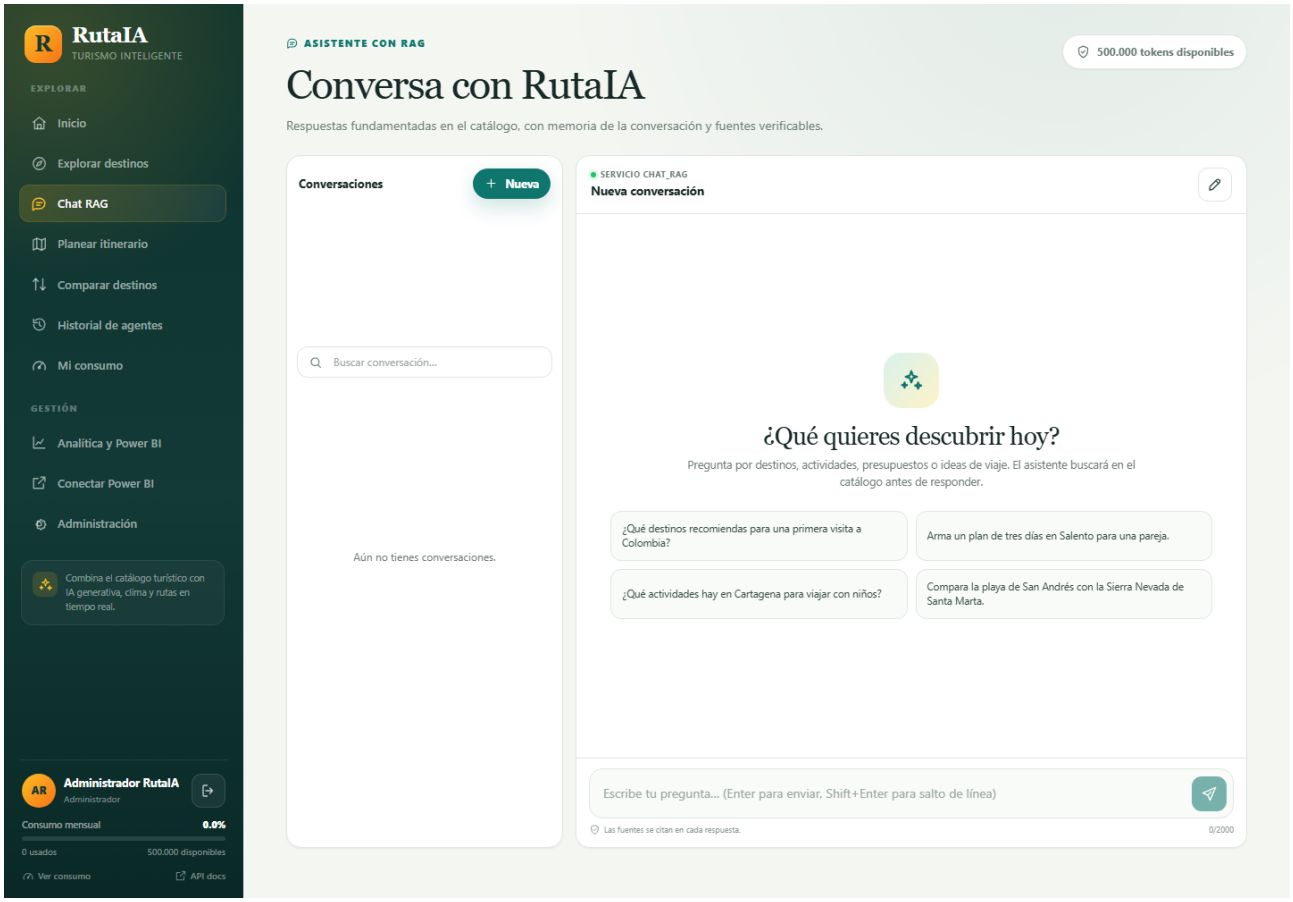
Inicio: catálogo, servicios activos, cuota y últimas ejecuciones



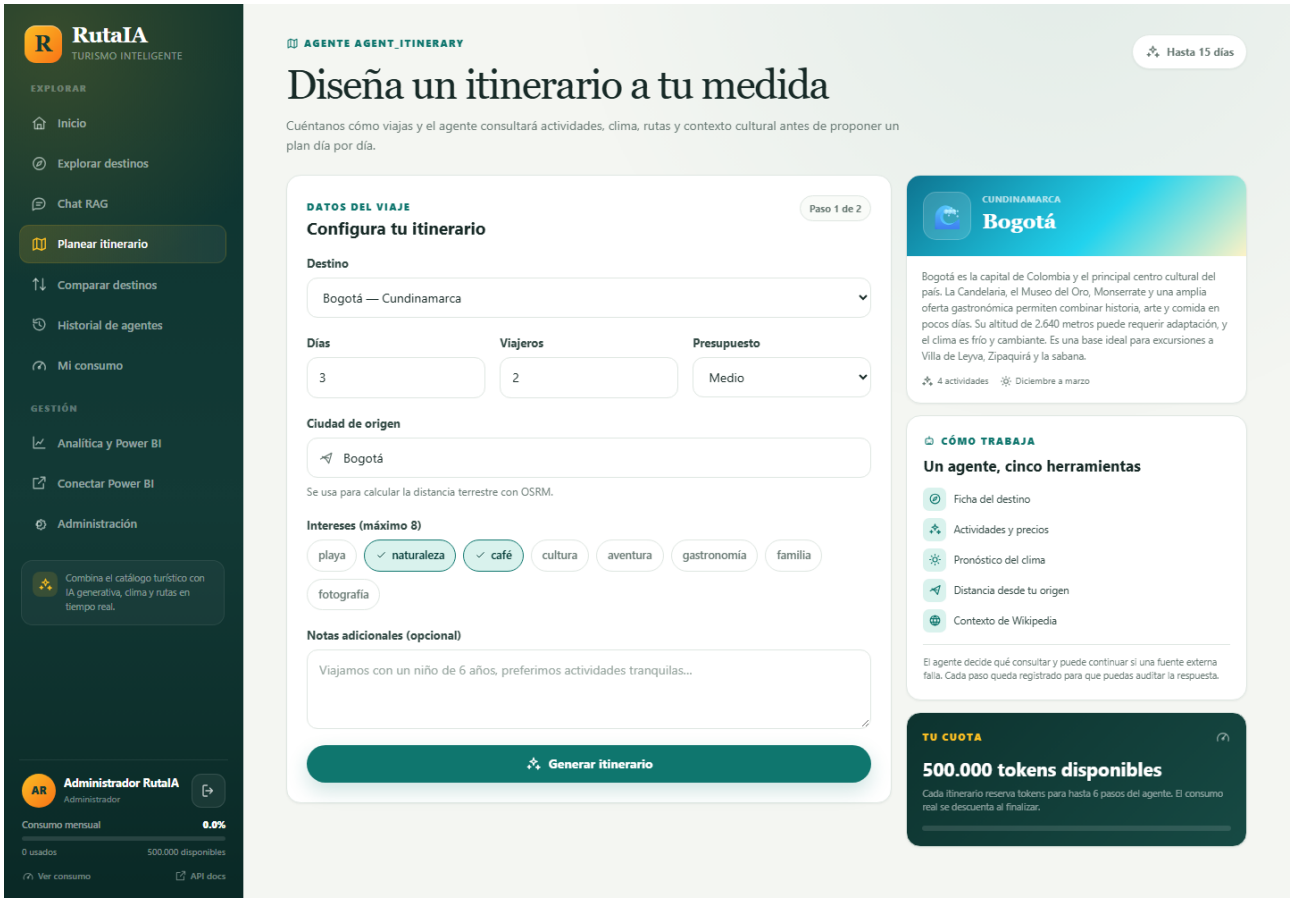
Catálogo con búsqueda, filtros y paginación



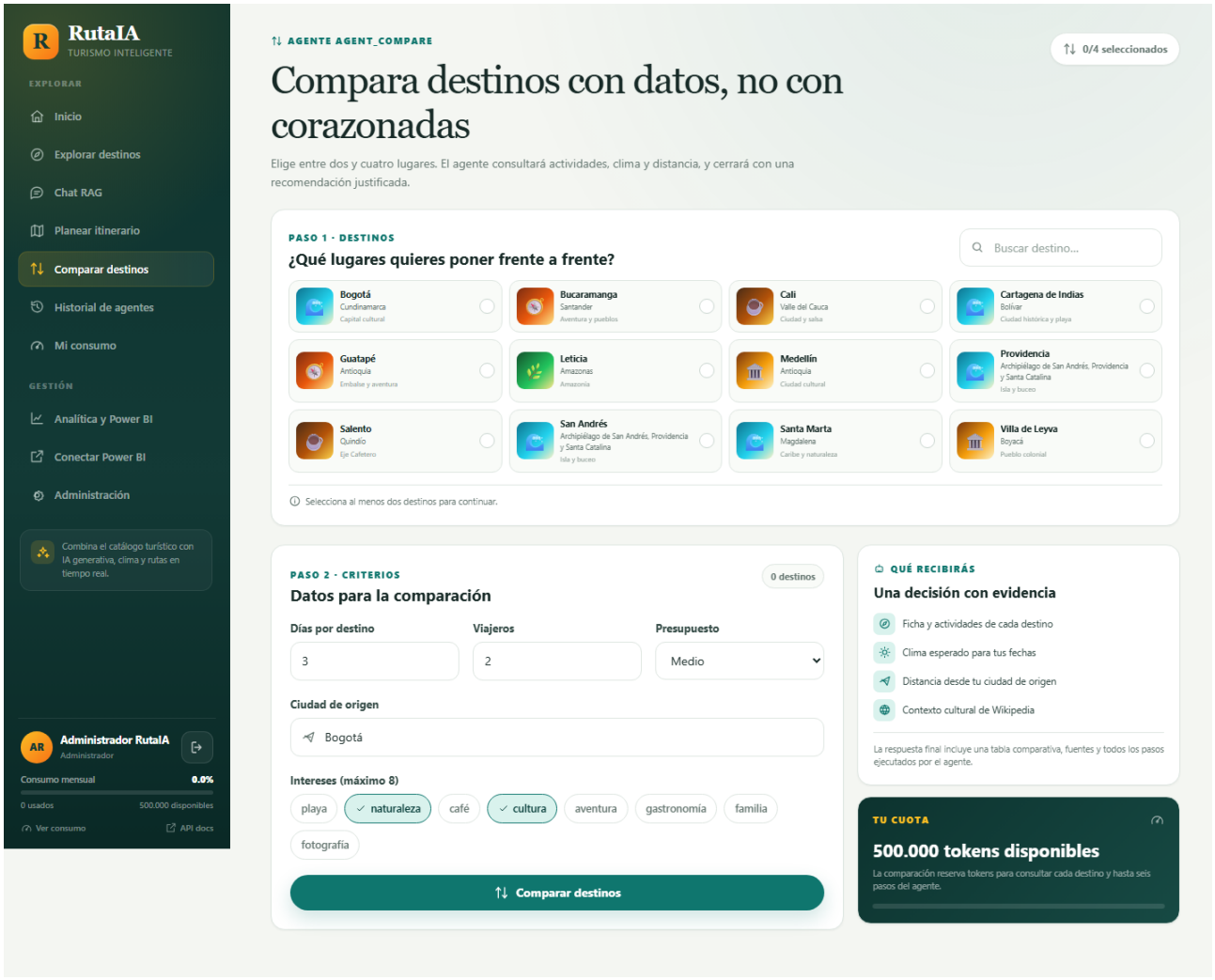
Ficha de destino con actividades y precios



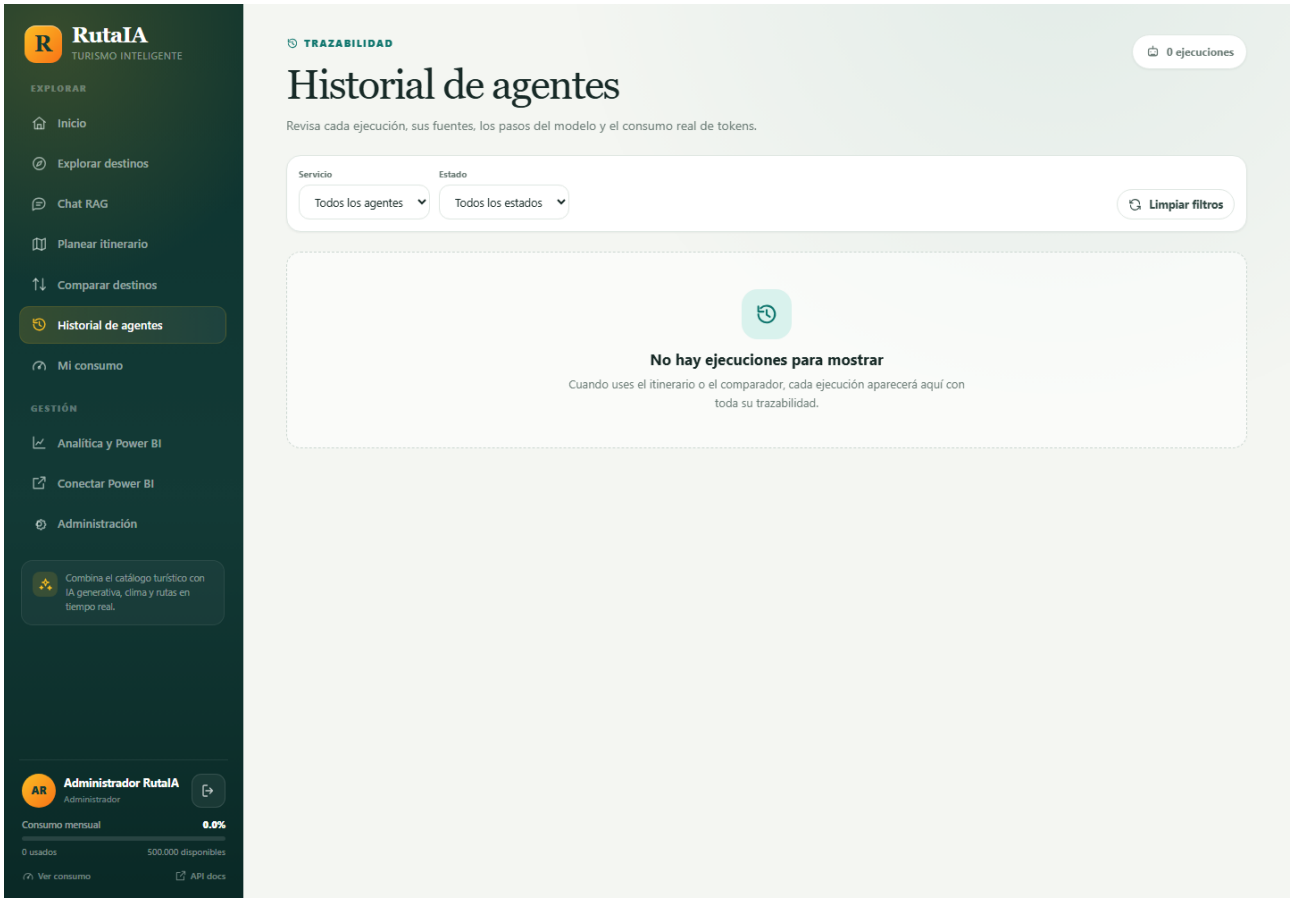
Chat RAG con sugerencias e historial



Agente de itinerarios: formulario e información de herramientas



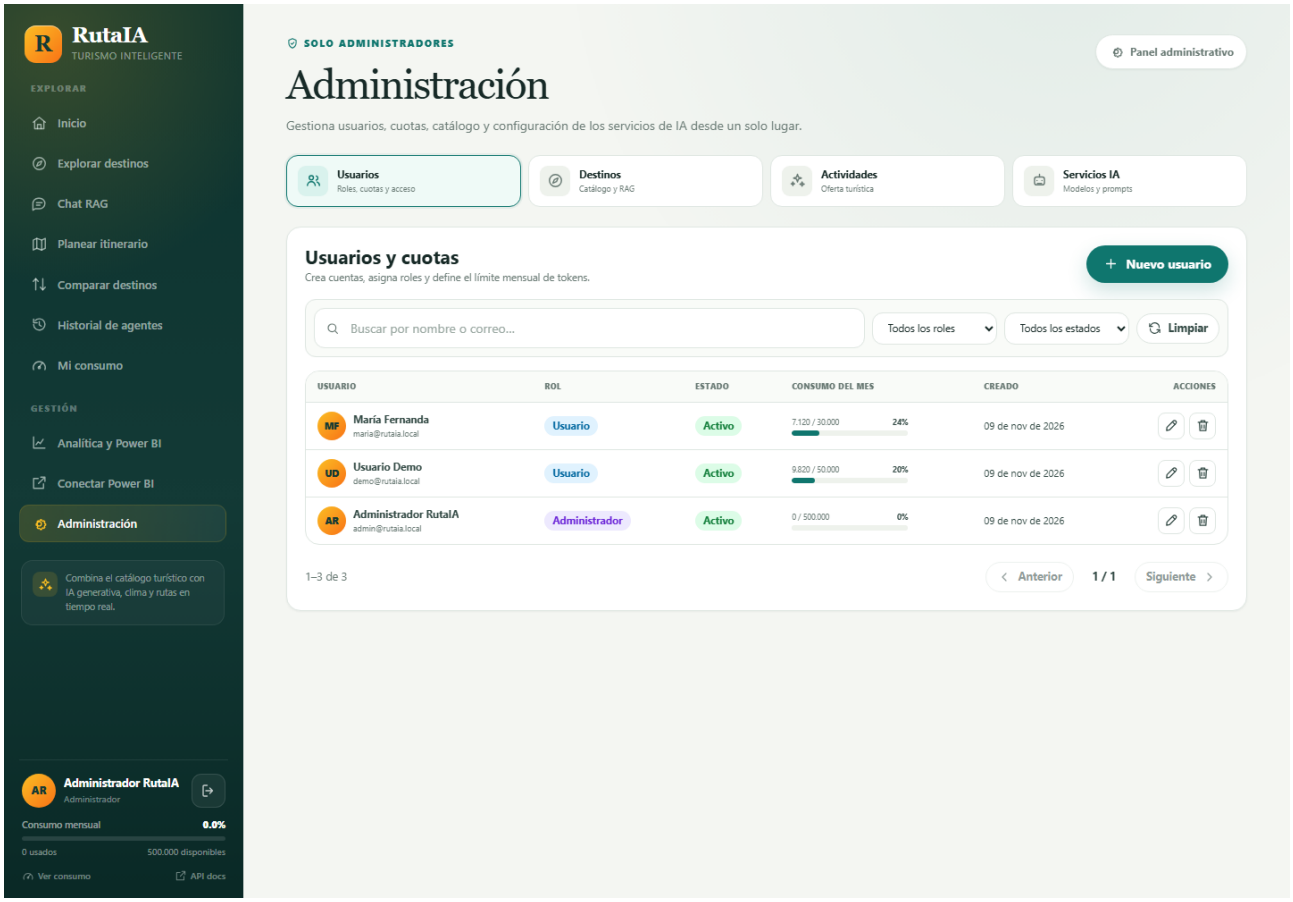
Comparador de destinos con selección múltiple



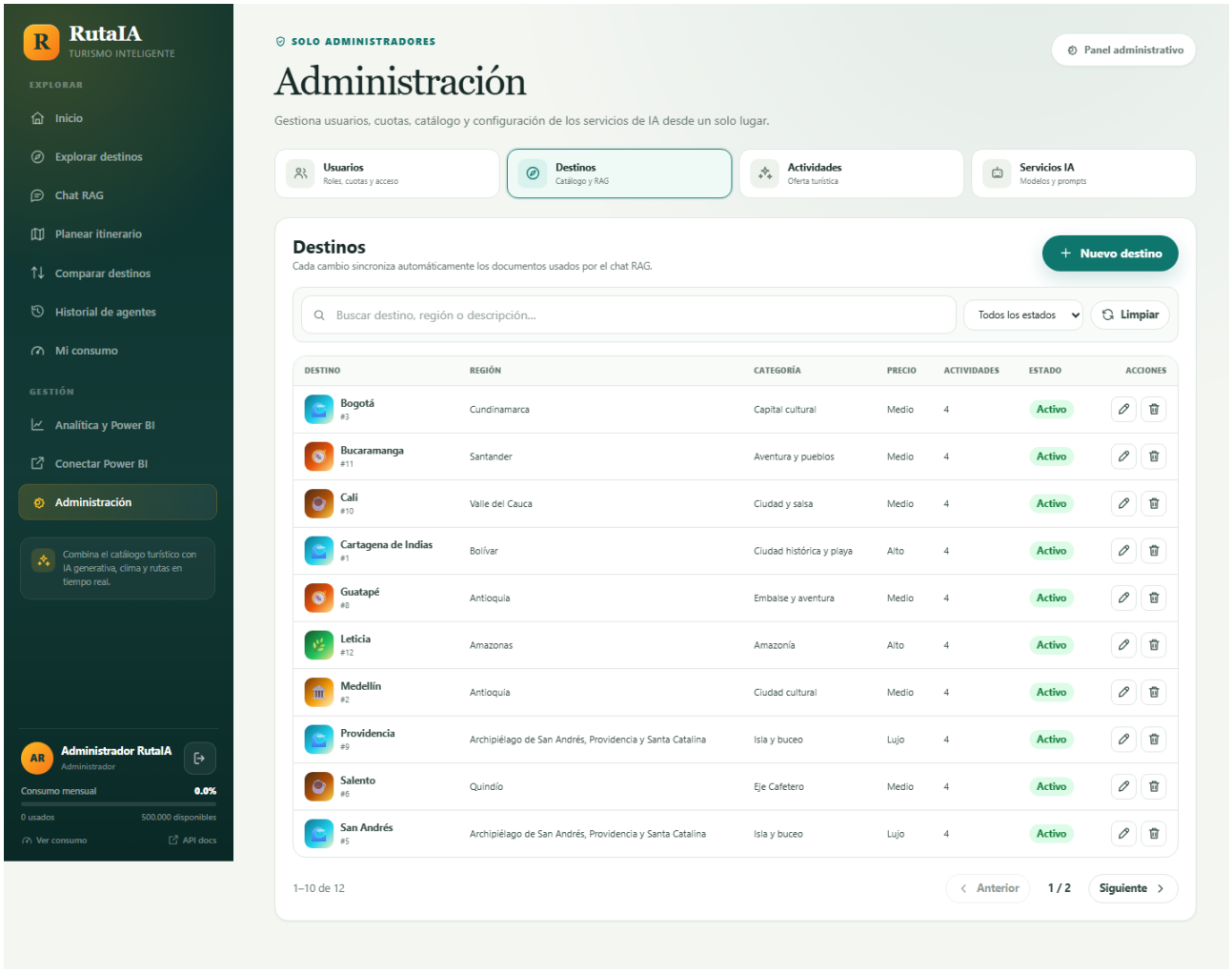
Historial de ejecuciones de agentes



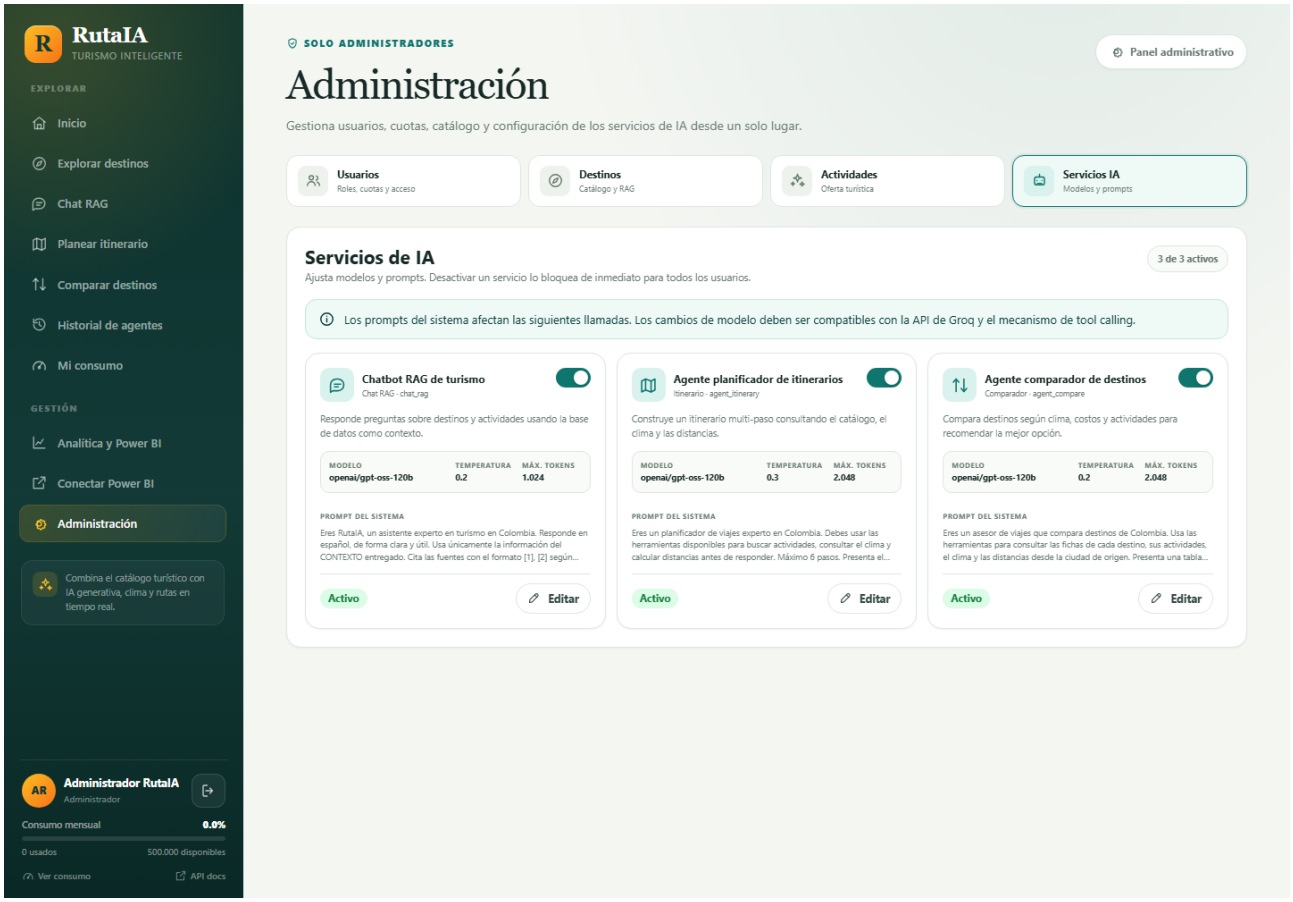
Portal de cuota de tokens



Administración de usuarios y cuotas



Administración de destinos



Configuración y activación de servicios de IA



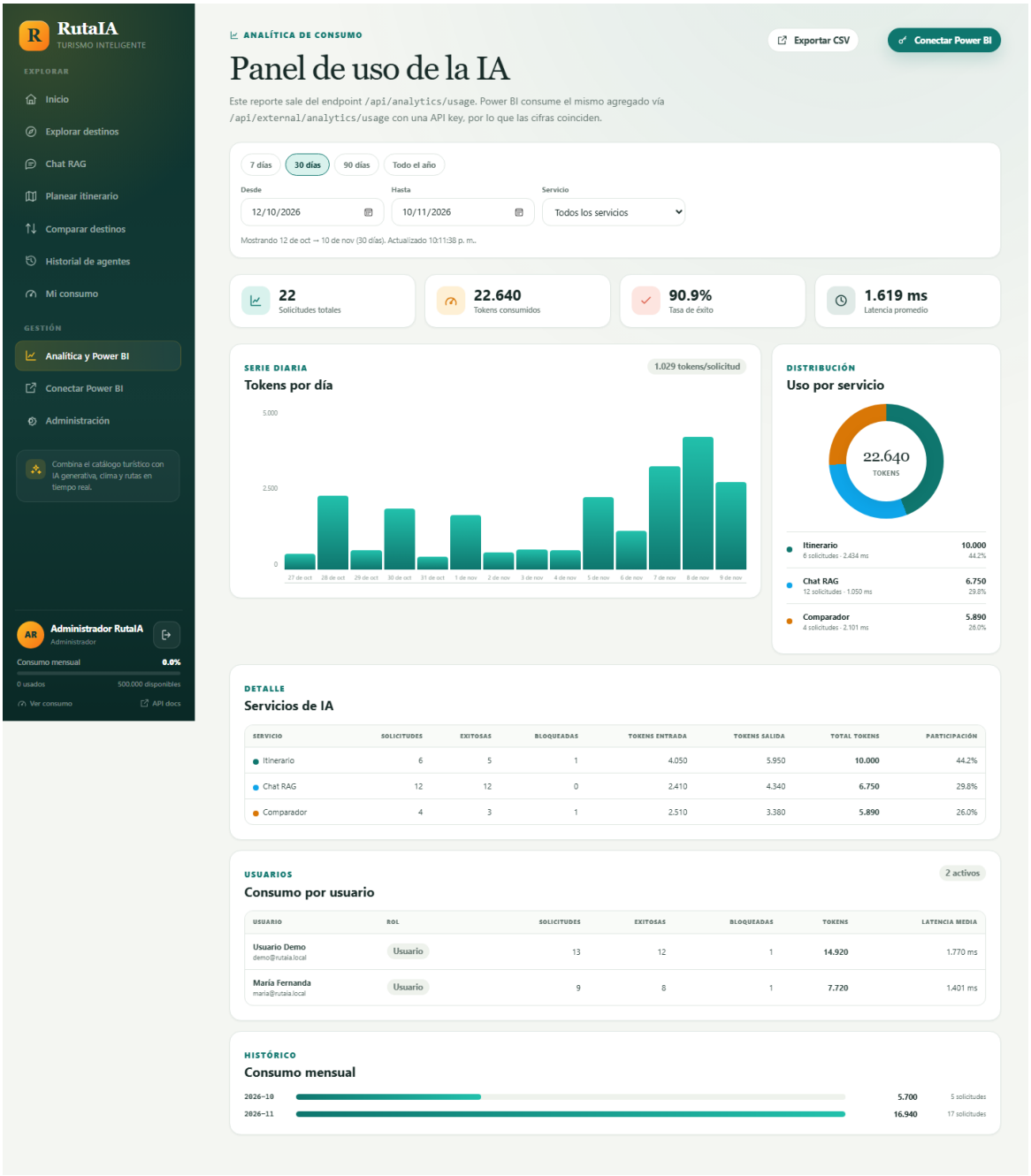
Vista móvil del chat



Vista móvil del inicio

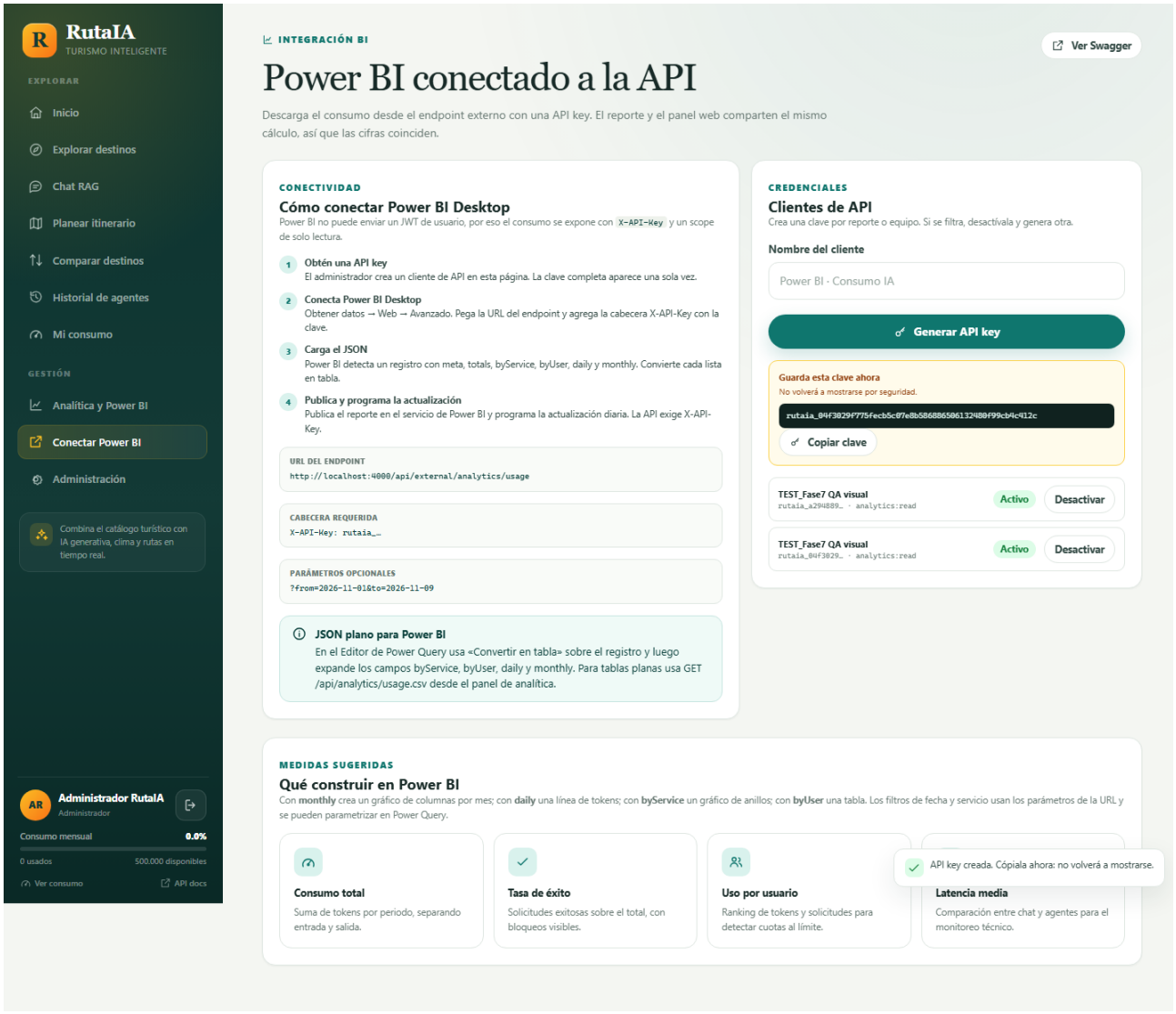
6. Analítica y Power BI

El panel web y Power BI consumen la misma función `getUsageAnalytics`. La única diferencia es la autenticación: JWT para el panel y `X-API-Key` para el endpoint externo. El reporte incluye totales, tasa de éxito, latencia, desglose por servicio y por usuario, serie diaria e histórico mensual.



Panel de analítica con los datos de demostración

Para el rango 11 de octubre a 9 de noviembre de 2026 el reporte devuelve 22 solicitudes, 22.640 tokens, 90,91 % de éxito y 1.619 ms de latencia promedio. Estas cifras son la referencia de validación del reporte de Power BI.



Alta de API keys e instrucciones de conexión para Power BI

Pasos para Power BI Desktop

- Crear una API key en Conectar Power BI; solo se muestra una vez.
- Obtener datos → Web → Avanzado y pegar la URL del endpoint externo.
- Añadir la cabecera `X-API-Key` con la clave.
- Convertir el registro en tabla y expandir byService, byUser, daily y monthly.
- Publicar en el servicio de Power BI y programar la actualización diaria.

```
GET https://rutaia-api.onrender.com/api/external/analytics/usage
    ?from=2026-10-11&to=2026-11-09
X-API-Key: rutaia_******************************
```

7. Calidad y pruebas

Las pruebas usan Vitest, Supertest y una base MySQL real. Groq y las APIs externas se simulan con mocks, de modo que la suite es determinista y no consume cuota. Cada archivo crea sus propios usuarios y limpia sus datos.

Archivo	Enfoque	Pruebas
api.test.js	Salud, errores, 404, rate limit y seguridad básica	20
fase3.test.js	Login, roles, usuarios y catálogo administrable	28
fase4.test.js	Chat RAG, memoria, fuentes y cuota	8
fase5.test.js	Agentes, herramientas, trazas y cuotas	9
fase7.test.js	Analítica, CSV, API keys y endpoint externo	10
Total	Vitest + Supertest sobre MySQL 8	68

La CI de GitHub Actions reejecuta las 68 pruebas desde cero contra MySQL 8.4, aplica esquema y seed, ejecuta la ingesta del RAG y arranca la API en modo producción. El workflow smoke.yml verifica la URL publicada y codeql.yml analiza la seguridad del código.

Comandos de verificación

```
cd server && npm test              # 68 pruebas de integración
cd client && npm run build         # build de producción
cd client && npm run lint          # oxlint (0 errores)
node scripts/smoke-api.mjs <url>   # health, login, catálogo y analítica
node scripts/qa-frontend.mjs       # capturas y detección de desbordes
```

Resultados obtenidos

- Backend: 5 archivos y 68 pruebas en verde (~8 s).
- Frontend: build de ~368 kB (102 kB gzip) y lint sin errores.
- Docker: imagen multipropósito con usuario sin privilegios y health check.
- CI: jobs de backend, frontend y despliegue en verde; el despliegue se omite con un aviso si no hay secretos.
- Base posterior a las pruebas: 3 usuarios, 12 destinos, 48 actividades y 60 documentos.

8. Demostración con Postman

La colección `docs/postman_collection.json` contiene 28 peticiones agrupadas en siete carpetas. Los scripts de prueba guardan automáticamente el JWT, la conversación, la ejecución y la API key en variables de colección. El entorno está en `docs/postman_environment.json`.

Carpeta	Contenido	Variables que produce
00 Sistema	Health check y OpenAPI	—
01 Autenticación	Login admin/demo, perfil y error 401	adminToken, userToken
02 Usuarios	Listado, alta, cuota y 403	—
03 Catálogo	Destinos, actividades y alta	destinationId
04 Chat RAG	Mensaje, memoria, historial y cuota	conversationId
05 Agentes	Itinerario, comparación, historial y detalle	runId
06 Analítica	Reporte, CSV, API key y reporte externo	apiKey

Orden recomendado: ejecutar 01 completo, luego 03 para obtener un destino, 04 para el chat y la cuota, 05 para los agentes y 06 para la analítica. La API key creada en la última carpeta alimenta el reporte externo de Power BI.

9. Despliegue y operación

Componente	Destino	Configuración
API	Render Web Service	render.yaml · build npm ci --omit=dev · start npm start · health /api/health
Frontend	Vercel (static)	client/vercel.json · VITE_API_URL · reescritura a index.html
Base de datos	MySQL 8 gestionado	db/schema.sql + db/seed.sql + npm run db:ingest
Imagen local	Docker	docker compose --profile full up -d --build
CI/CD	GitHub Actions	ci.yml, codeql.yml, smoke.yml y Dependabot

El pipeline se activa en cada push y pull request a main. Solo el push a main ejecuta el job de despliegue, y este se salta de forma controlada si faltan los secretos de Render o Vercel. El detalle de secretos y variables está en `docs/deployment.md`.

Variables de entorno

Variable	Ámbito	Descripción
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	API	Conexión MySQL
JWT_SECRET, JWT_EXPIRES_IN	API	Firma y vigencia del JWT
GROQ_API_KEY, GROQ_MODEL, GROQ_MODEL_LITE	API	Proveedor de IA y modelos
CORS_ORIGIN	API	Orígenes permitidos (coma separada)
TOKEN_QUOTA_DEFAULT	API	Cuota inicial de nuevos usuarios
VITE_API_URL	Frontend	URL pública de la API

10. Seguridad, limitaciones y trabajo futuro

- Contraseñas con bcrypt y JWT firmados; el token se valida contra el estado actual del usuario.
- Las API keys se guardan como hash SHA-256 y se muestran una única vez.
- Consultas SQL parametrizadas, validación Zod y límite de intentos de login.
- Helmet, CORS restringido, rate limit y cabecera X-Request-Id en cada respuesta.
- El system_prompt de los servicios solo se expone a administradores.
- La GROQ_API_KEY vive únicamente en el backend.

Limitaciones conocidas

- La cuota se controla con una reserva estimada; dos solicitudes simultáneas podrían competir por el último margen.
- El RAG usa recuperación léxica FULLTEXT; no hay embeddings porque Groq no expone esa API.
- Wikipedia puede limitar solicitudes frecuentes; la caché y el manejo de errores de herramientas lo mitigan.
- El plan gratuito de un proveedor puede suspender la API por inactividad.

Trabajo futuro

- Reserva atómica de cuota dentro de una transacción y conciliación posterior.
- Embeddings con un proveedor dedicado y búsqueda híbrida.
- Panel de auditoría de API keys con rotación programada.
- Reservas y pagos integrados con el catálogo.
- Pruebas end-to-end de navegador en la CI con Playwright.

Repositorio y enlaces

```
Repositorio: https://github.com/FerMar707501/PFinalTurismo
API local:   http://localhost:4000
Swagger:     http://localhost:4000/api/docs
Frontend:    http://localhost:5173
Power BI:    https://rutaia-api.onrender.com/api/external/analytics/usage
```

Documento generado a partir del código y de las capturas verificadas del repositorio. Las cifras de analítica corresponden a los datos de demostración del seed, por lo que son reproducibles con la misma base.